makes use of the injector method to inject dependencies. The test to be conducted is given in *it( )*. The test suite is *describe( )*, and both *beforeEach( )* and *it( )* come inside it. E2E testing makes use of all the above functions. One other function used is *expect( )*. This creates 'expectations', which verify if the particular application's state (value of a variable or URL) is the same as the expected values.

Recommended frameworks for unit testing are Jasmine and Karma, and for E2E testing, Protractor is the one to go with.

Who uses AngularJs?

Some of the following corporate giants use AngularJS:

- Google
- Sony (YouTube on PS3)
- Virgin America
- Nike
- msnbc (msnbc.com)

You can find a lot of interesting and innovative apps in the 'Built with AngularJS' page.

Competing technologies

Features	Ember.js	AngularJS	Backbone.js
Routing	Yes	Yes	Yes
Views	Yes	Yes	Yes
Two-way binding	Yes	Yes	No

The chart above covers only the core features of the three frameworks. Angular is the oldest of the lot and has the biggest community. END

References

[1] *http://singlepageappbook.com/goal.html*
[2] *https://github.com/angular/angular.js*
[3] *https://docs.angularjs.org/guide/*
[4] *http://karma-runner.github.io/0.12/index.html*
[5] *http://viralpatel.net/blogs/angularjs-introduction-hello-world-tutorial/*
[6] *https://builtwith.angularjs.org/*

By: Tina Johnson

The author is a FOSS enthusiast who has contributed to Mediawiki and Mozilla's Bugzilla. She is also working on a project to build a browser (using AngularJS) for autistic children.

To be continued from page.... 37

There are some statements like condition checking where *'f b1'* can be computed even without requiring the subsequent arguments, and hence the *foldr* function can work with infinite lists. There is also a strict version of *foldl (foldl')* that forces the computation before proceeding with the recursion.

If you want a reference to a matched pattern, you can use the *as* pattern syntax. The tail function accepts an input list and returns everything except the head of the list. You can write a *tailString* function that accepts a string as input and returns the string with the first character removed:

```
tailString :: String -> String
tailString "" = ""
tailString input@(x:xs) = "Tail of " ++ input ++ " is " ++ xs
```

The entire matched pattern is represented by *input* in the above code snippet.

Functions can be chained to create other functions. This is called 'composing' functions. The mathematical definition is as under:

```
(f o g)(x) = f(g(x))
```

This dot (.) operator has the highest precedence and is left-associative. If you want to force an evaluation, you can use the function application operator (**$**) that has the second highest precedence and is right-associative. For example:

```
*Main>  (reverse ((++) "yrruC " (unwords ["skoorB",
"lleksaH"])))
"Haskell Brooks Curry"
```

You can rewrite the above using the function application operator that is right-associative:

```
Prelude> reverse $ (++) "yrruC " $ unwords ["skoorB",
"lleksaH"]
"Haskell Brooks Curry"
```

You can also use the dot notation to make it even more readable, but the final argument needs to be evaluated first; hence, you need to use the function application operator for it:

```
*Main> reverse . (++) "yrruC " . unwords $ ["skoorB",
"lleksaH"]
"Haskell Brooks Curry"
```

END

By: Shakthi Kannan

The author is a free software enthusiast and blogs at *shakthimaan.com*.